



Machine Learning & Deep Learning

Dr. André Artelt, MSc. Inaam Ashraf, MSc. Luca Hermes

{aartelt,mashraf,lhermes}@techfak.uni-bielefeld.de

WDSA CCWI 2026

Paphos, 18/05/2026



Water-Futures has received funding from the European Research Council (ERC) under the European Union's Horizon 2020 research and innovation programme (Grant agreement No. 951424)

This research was supported by the Ministry of Culture and Science NRW (Germany) as part of the Lamarr Fellow Network. This publication reflects the views of the authors only.

Outline of Today

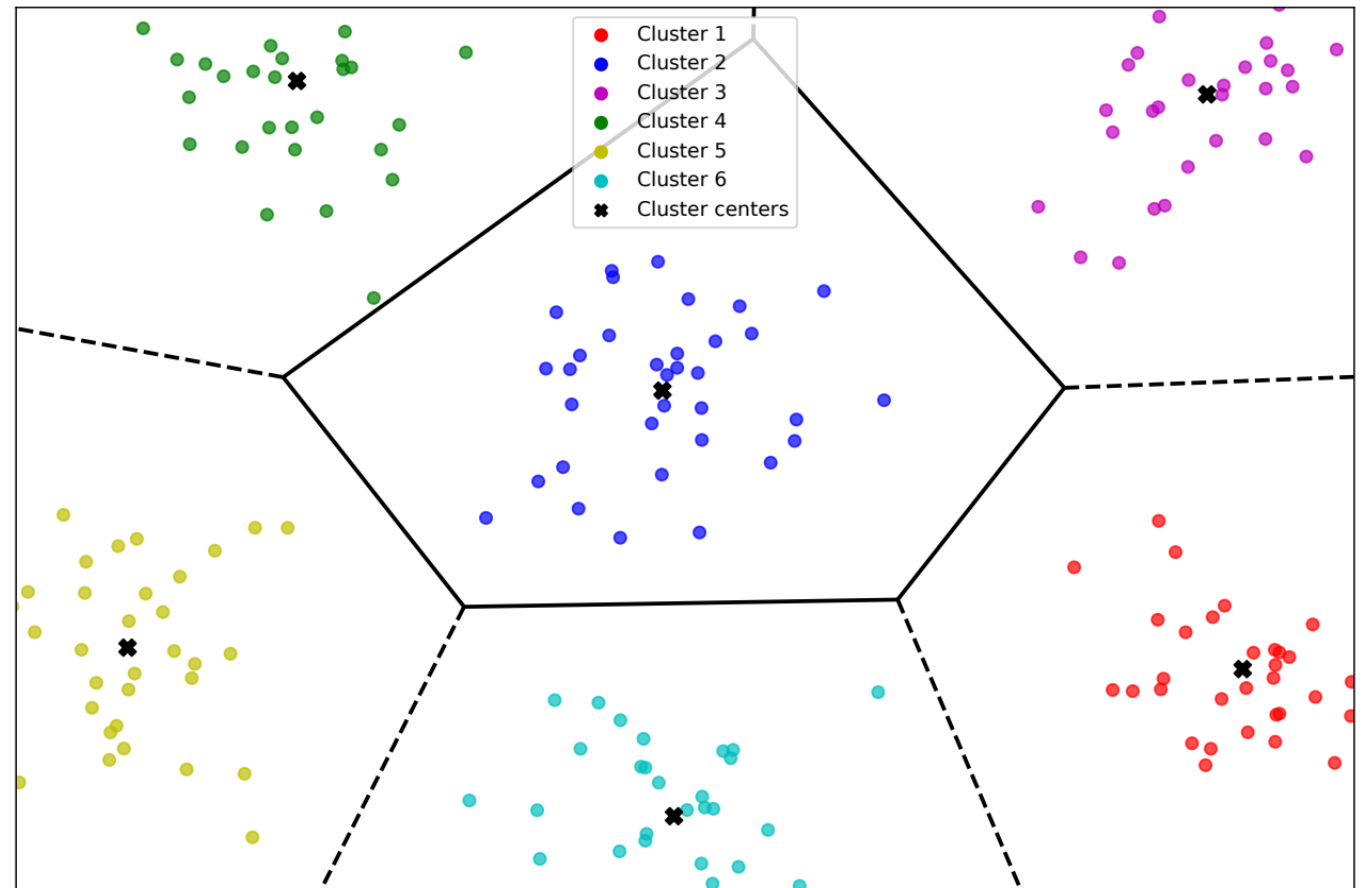
Time	Topics
14:00	Introduction & Welcome
	Surrogate models vs. digital twins
	Types & Implementations of surrogates
14:15	Recap of Machine Learning (ML) and Deep Learning
	Basic concepts, evaluation, hyper-parameter tuning, etc.
	Surrogate models: Challenges for ML
	Data-driven mitigation strategies (i.e., scenario generation)
14:45	Physics-informed/inspired Methods -- Part I
	Intro to Physics-informed Neural Networks (PINNs)
	Hands-On: How to build and train a PINN in Python
15:30	Coffee Break
16:00	Physics-informed/inspired Methods -- Part II
	Operators vs. PINNs
16:25	Case study: SOTA Hydraulic Surrogate Model
	Hydraulic surrogates and their applications
	Analyzing the problem and identifying relevant quantities and laws
	A SOTA Hydraulic Surrogate
	Hands-On: Usage and Evaluation of the Surrogate in Python
17:15	Closing, Summary, and Q&A
17:30	END



We are here!

Basic Concepts

- Machine Learning = Automatically build parameterized models from data
- Different tasks:
 - Regression
 - Classification
 - Control
 - Dimensionality reduction
 - Clustering
 -



Basic Concepts

- Different types of data:
 - Sample or time series
 - Labeled or unlabeled data
 - Interactive environment (for control)
- Learning paradigms:
 - *Supervised* for labeled data
 - *Unsupervised* for unlabeled data
 - *Reinforcement* (control)

Basic Concepts – "The Pipeline"

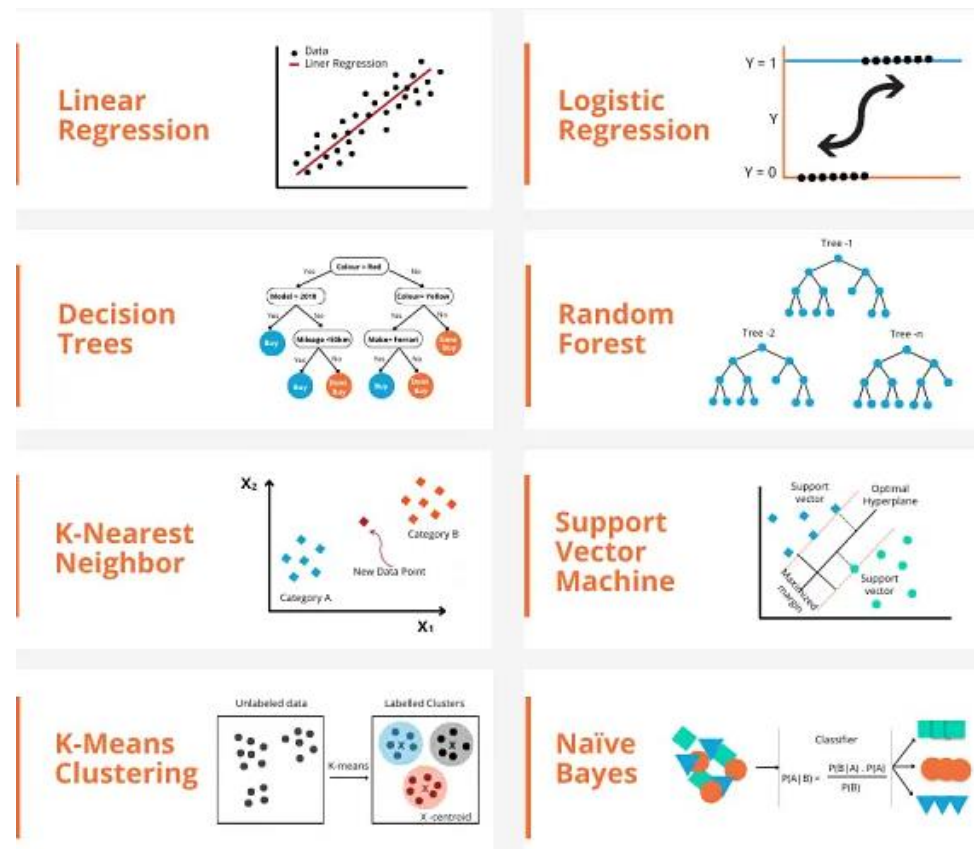
- Preprocessing
 - Dealing with missing features (e.g., imputations)
 - Scaling the input
- Loss minimization
 - Learning as an optimization problem

$$\arg \min_{\vec{w} \in \mathbb{R}^d} \left(- \sum_i y_i \log (\text{sgd}(\vec{w}^\top \vec{x}_i)) + (1 - y_i) \log (1 - \text{sgd}(\vec{w}^\top \vec{x}_i)) \right)$$

- Evaluation
 - Test generalization of learned model
 - Different metrics: Accuracy, mean-squared-error, etc.
 - Test for statistical significance compared to baselines!
 - Train-Test split (e.g., cross-validation)

Machine Learning (ML) Models

- Many different types of ML models exist
 - Different characteristics, assumptions, and benefits
- => Pick your model wisely!

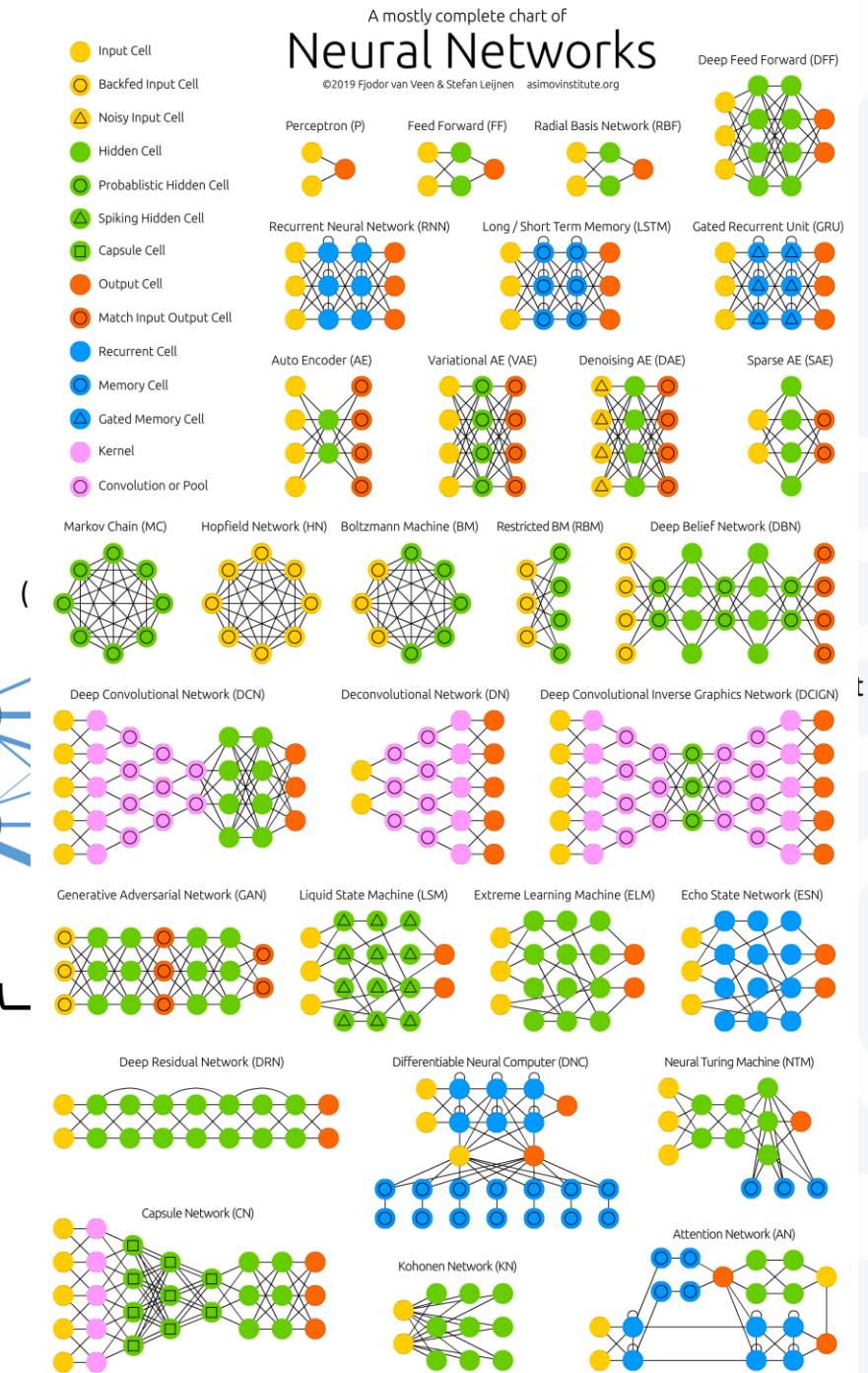


(Deep) Neural Networks

- Artificial Neural Networks (ANNs) can learn complex patterns for all tasks:
 - Classification
 - Regression
 - Control
- Concatenation of linear mappings + non-linear activation function

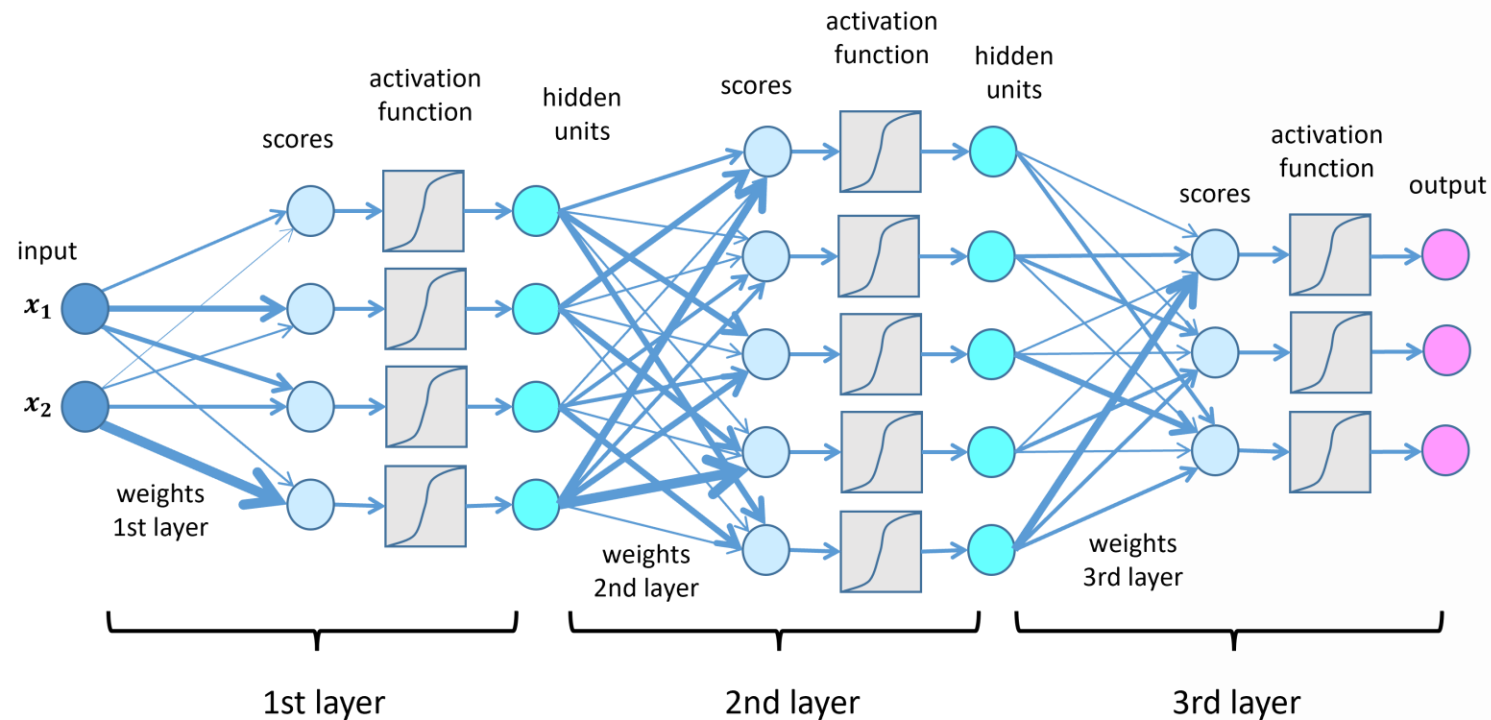
$$\sigma \left(\mathbf{W}_2 \sigma \left(\mathbf{W}_1 \vec{x} + \vec{b}_1 \right) + \vec{b}_2 \right)$$

- Many different architectures!
=> *Tailored for different problems*



(Deep) Neural Networks

- "Deep" Neural Networks = "Many" hidden layers



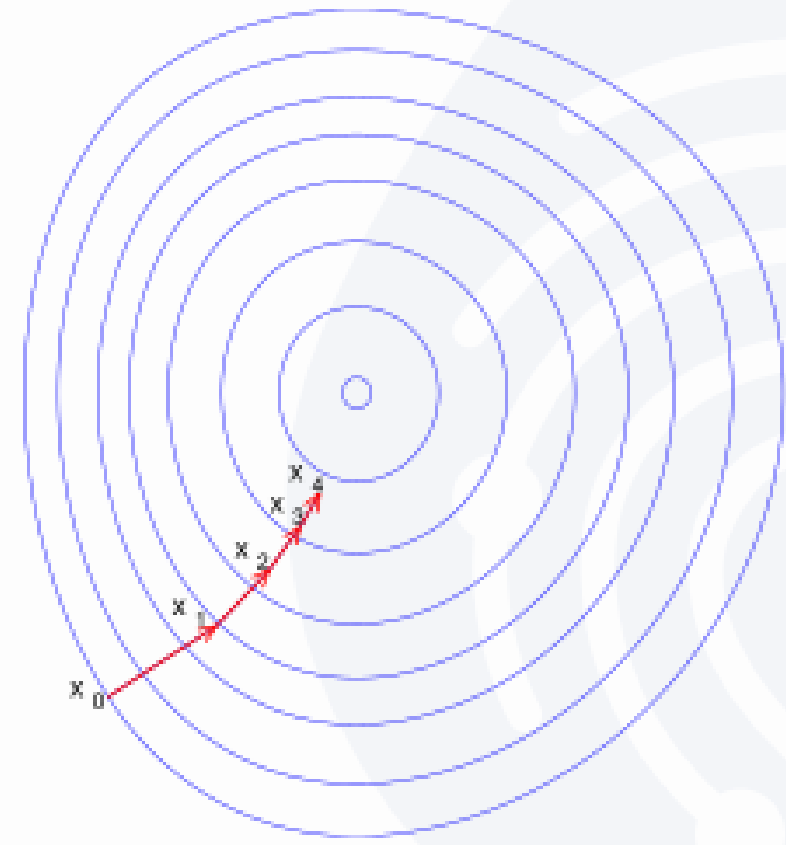
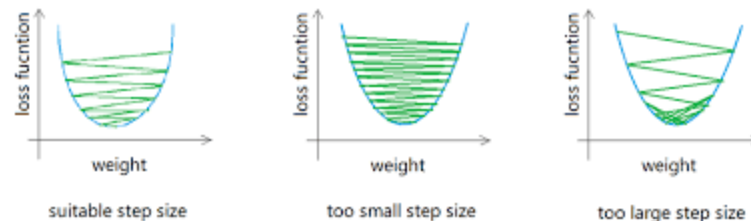
<https://lamarr-institute.org/blog/deep-neural-networks/>

(Deep) Neural Networks

- Gradient-based optimization => minimize loss function

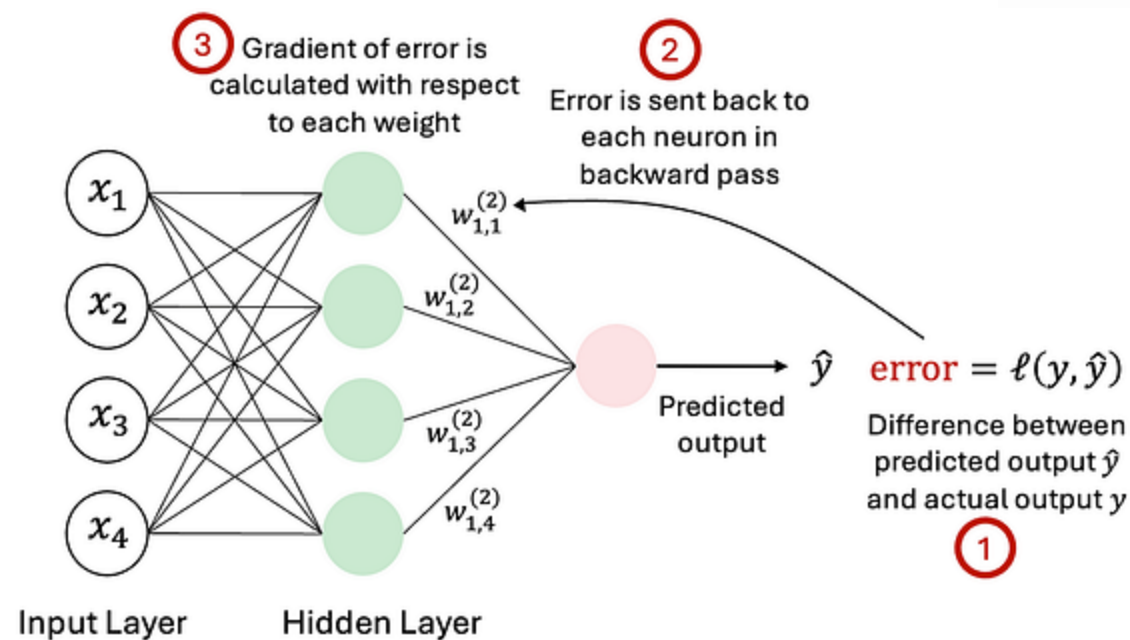
$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$$

- Gradient descent (batch or stochastic):
 - Iteratively towards an optimum
 - Step-size (small but not too small)
 - Gradient clipping (norm scaling)
 -



Optimization via Gradient Descent

- Backpropagation for efficient computation of the gradients



<https://medium.com/@lmpo/backpropagation-the-backbone-of-neural-network-training-64946d6c3ae5>

Challenges for ML-based (Surrogate) Models



- Correlations & biases => Overfitting
- Uncertainties => Underfitting
-

=> How can we deal with those?



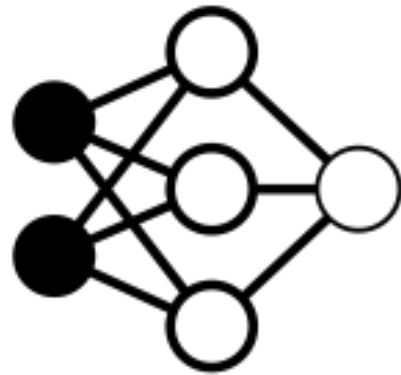
European Research Council
Established by the European Commission

Challenges: Overfitting

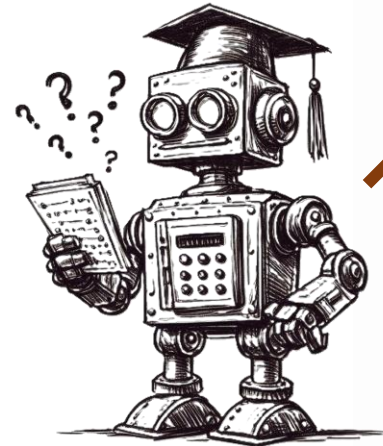


Data

+

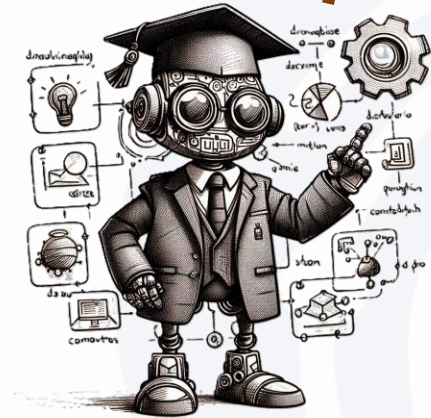
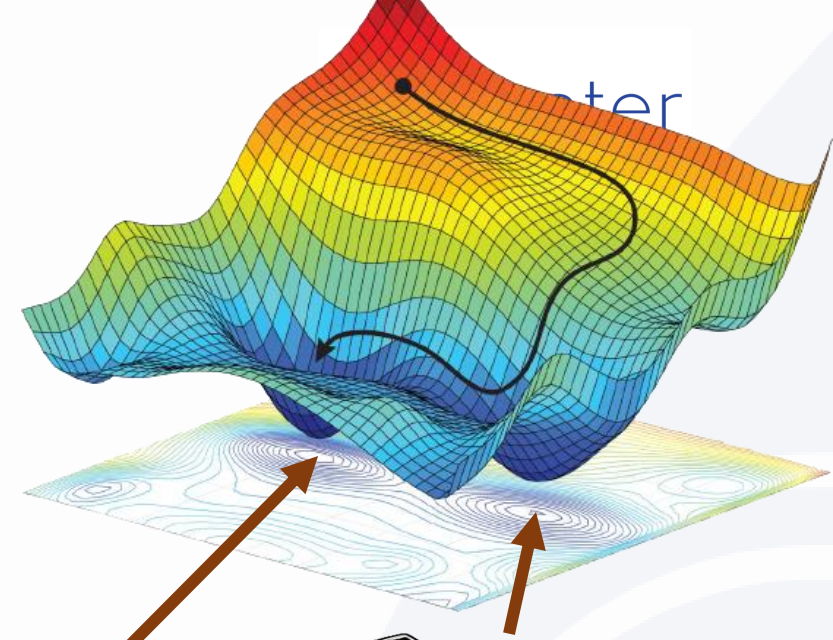


=



Solution 1

- Matches the Training data
- But not the Testing data
→ Bad generalization!

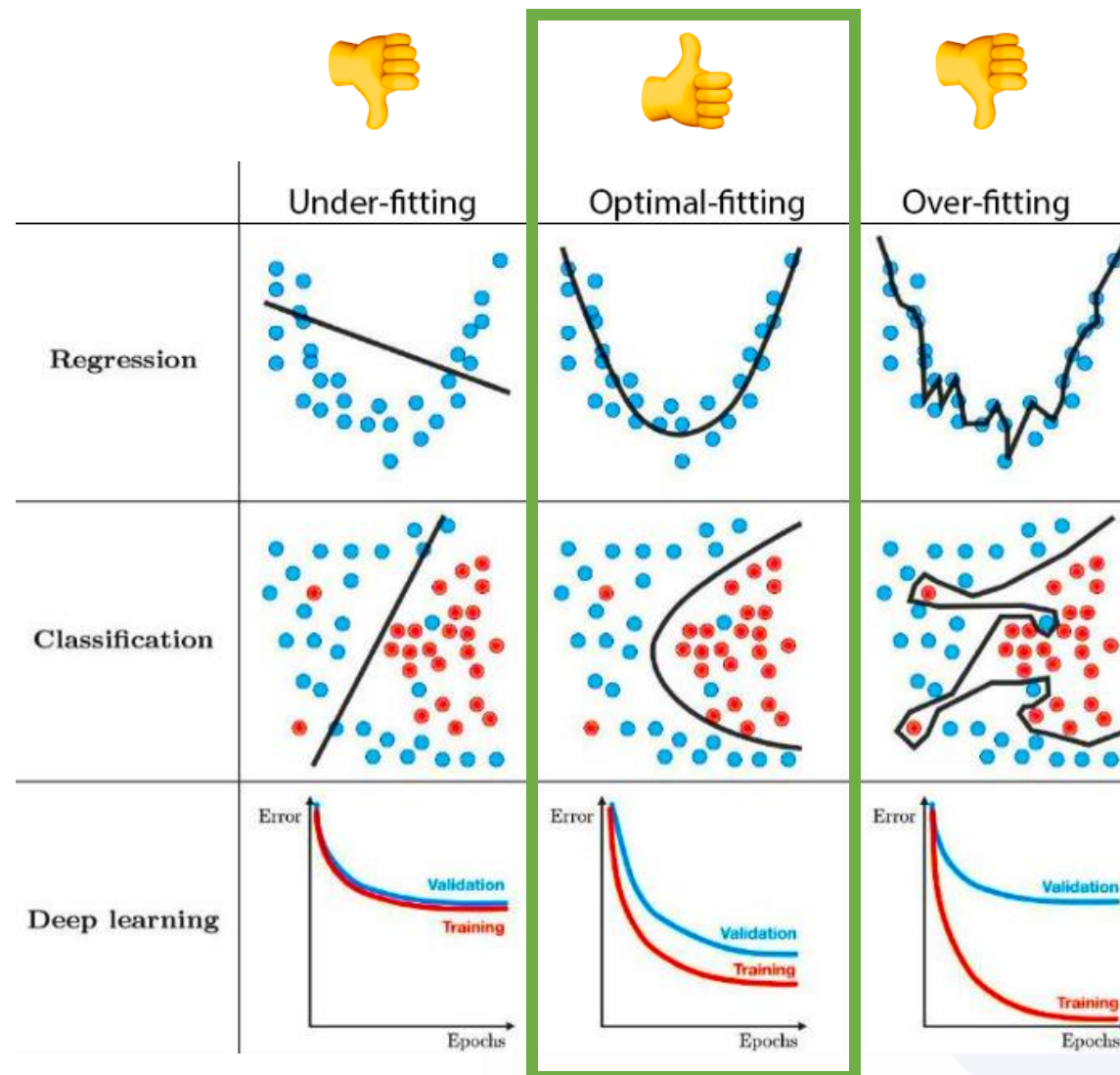


Solution 2

- Matches both Training and Testing data
→ Good generalization!

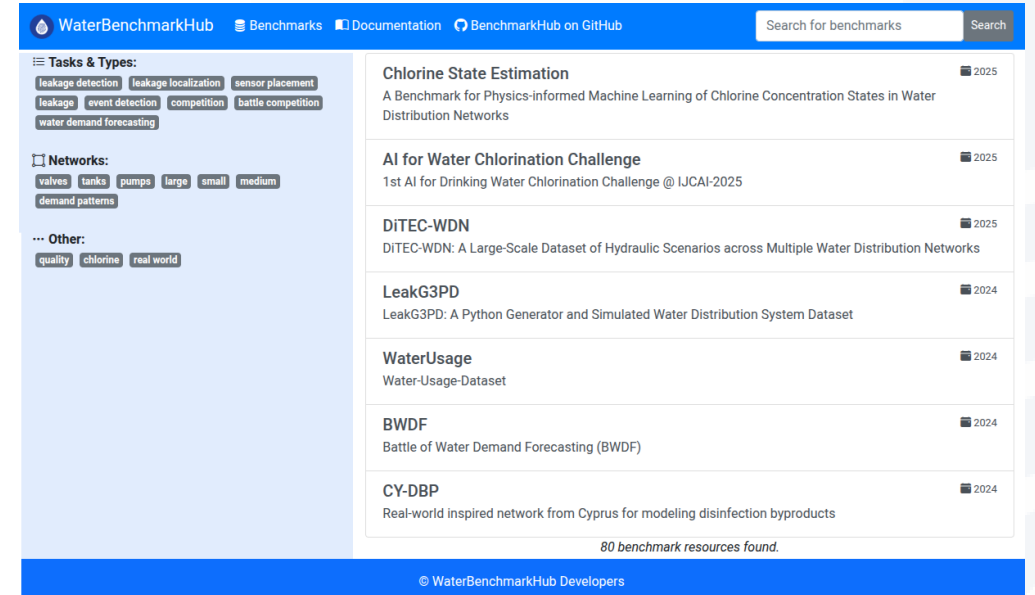
Challenges -- Over-/Underfitting

- *How do we find a good fit?*
 - Regularization & Inductive Biases
 - More or better data



"High-quality" Training & Test Data

- Ensure "diverse" scenario generation
 - Recycle existing benchmarks
- => Check-out the [WaterBenchmarkHub](https://waterbenchmarkhub.org/)



The screenshot shows the WaterBenchmarkHub website interface. The header includes the site name and navigation links for Benchmarks, Documentation, and GitHub. A search bar is present on the right. The left sidebar contains filters for Tasks & Types (leakage detection, leakage localization, sensor placement, leakage, event detection, competition, battle competition, water demand forecasting) and Networks (valves, tanks, pumps, large, small, medium, demand patterns). The main content area lists several benchmarks with their descriptions and years:

Benchmark Name	Description	Year
Chlorine State Estimation	A Benchmark for Physics-informed Machine Learning of Chlorine Concentration States in Water Distribution Networks	2025
AI for Water Chlorination Challenge	1st AI for Drinking Water Chlorination Challenge @ IJCAI-2025	2025
DITEC-WDN	DITEC-WDN: A Large-Scale Dataset of Hydraulic Scenarios across Multiple Water Distribution Networks	2025
LeakG3PD	LeakG3PD: A Python Generator and Simulated Water Distribution System Dataset	2024
WaterUsage	Water-Usage-Dataset	2024
BWDF	Battle of Water Demand Forecasting (BWDF)	2024
CY-DBP	Real-world inspired network from Cyprus for modeling disinfection byproducts	2024

80 benchmark resources found.

© WaterBenchmarkHub Developers

"High-quality" Training & Test Data

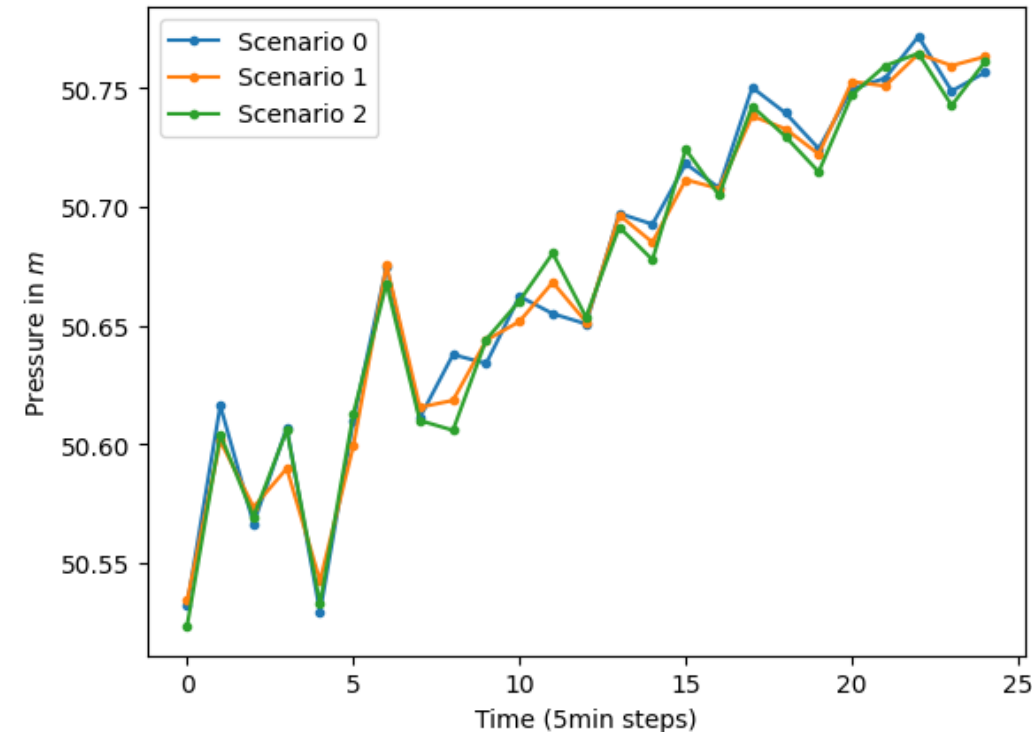
- Ensure "diverse" scenario generation
 - Recycle existing benchmarks
 - => Check-out the [WaterBenchmarkHub](#)
- **Break (unwanted) correlation**
 - Randomized demands
 - We want to learn the physics but do not overfit to demand patterns!

```
# Create new scenario based on Net1
with ScenarioSimulator(scenario_config=load_net1()) as sim:
    # Randomize demands
    sim.randomize_demands()

    # Run simulation
    scada_data = sim.run_simulation()
```


"High-quality" Training & Test Data

- Ensure "diverse" scenario generation
 - Recycle existing benchmarks
 - => Check-out the [WaterBenchmarkHub](https://waterbenchmarkhub.org/)
- **Break (unwanted) correlation**
 - Randomized demands
 - We want to learn the physics but do not overfit to demand patterns!
 - Add uncertainties & perturbations
 - Demand
 - Network topology (e.g., pipe diameter, length, etc.)



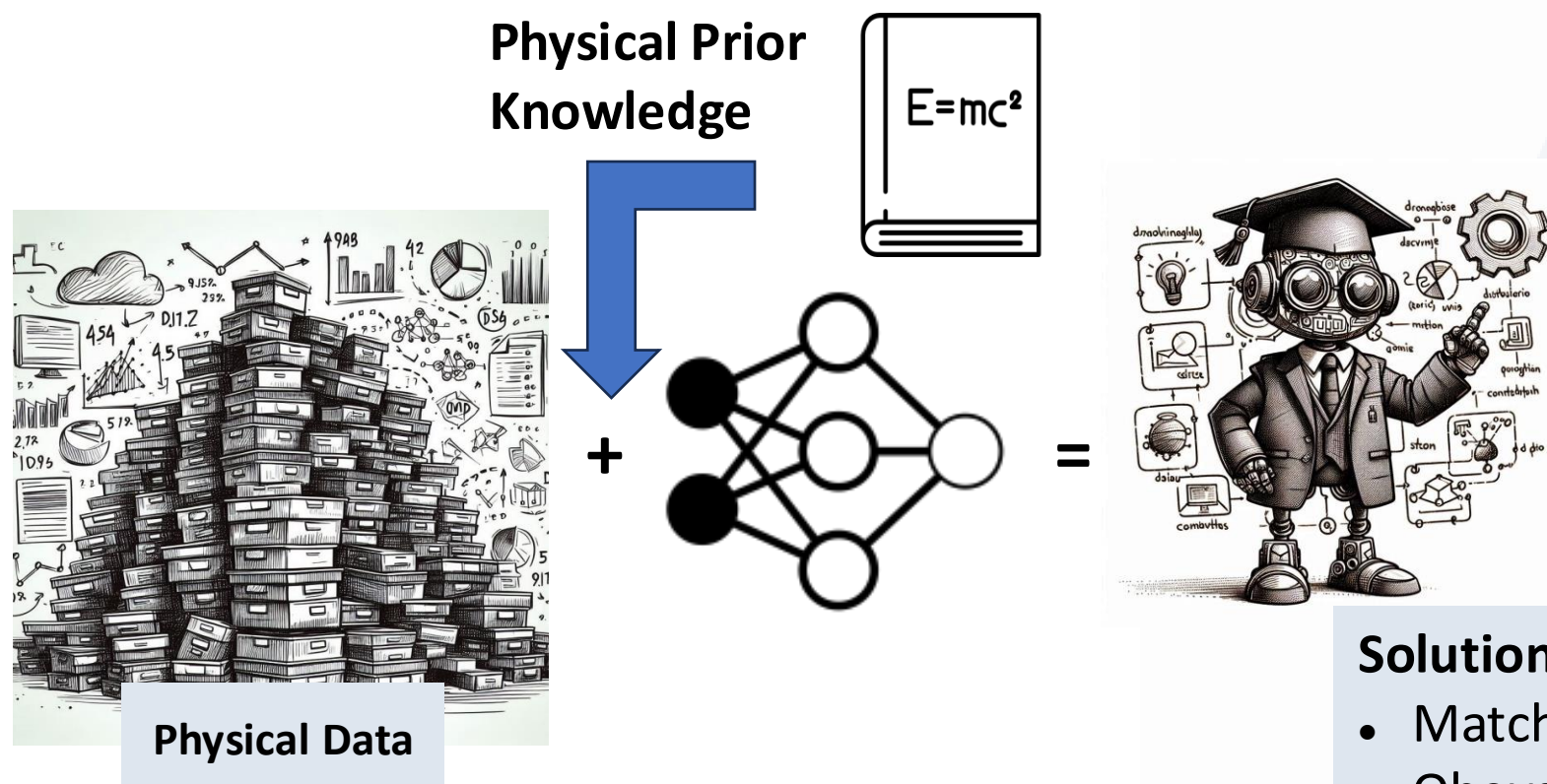
```
uc = RelativeUniformUncertainty(low=0.75, high=1.25)
```

```
sim.set_model_uncertainty(ModelUncertainty(global_demand_pattern_uncertainty=uc))
```

<https://epyt-flow.readthedocs.io/en/stable/examples/uncertainties.html>

What else?

- Inductive bias & regularization
 - Inject physics directly into the training process (e.g. loss function, architecture, etc.)



Solution

- Matches the data
- Obeys the Physics
- → Good generalization!



Next: Physics-informed/inspired Methods